Functional Programming: Paper Mock 1

Weeks 1-5 · 50 marks · 75 minutes · answer on paper, no computer

Written in the style of paper exams on this material at other universities: give types, evaluate expressions, write functions by hand, use data types, and prove a property by induction. Your module's own exam format hasn't been announced; check with the module team.

Write Haskell exactly as you would in a file: type signatures included, careful with brackets and indentation. You may use Prelude functions (`map`, `filter`, `foldr`, `zip`, `takeWhile`, `dropWhile`, `length`, `maximum`, ...) unless a question says otherwise. Answers are in `paper-1-answers`; try every question first.

Question 1: Types (8 marks, 1 each)

Give the most general type of each expression, including any class constraints.

a) `\x -> (x, x)`

b) `map (+1)`

c) `filter even`

d) `\f x -> f (f x)`

e) `zip "ab"`

f) `(.) not`

g) `head . tail`

h) `\x y -> x == y && x < 3`

Question 2: Evaluation (8 marks, 1 each)

Write the value of each expression. For (a), (b) and (g), show the steps.

a) `foldr (-) 0 [5, 3, 1]`

b) `foldl (-) 0 [5, 3, 1]`

c) `[x * y | x <- [1, 2], y <- [x .. 3]]`

d) `takeWhile (/= ' ') "fp exam"`

e) `zip [1 ..] "abc"`

f) `map (\(a, b) -> a * b) [(1, 2), (3, 4)]`

g) `f [1, 2, 3]`, where

```haskell
f :: [Int] -> Int
f []     = 0
```

```
f (x:xs) = x + 2 * f xs
```

h) `concatMap show [1, 22, 3]`

Question 3: Lists and recursion (10 marks)

Use recursion with pattern matching. Do not use list comprehensions in this question.

a) [3 marks] Write `count :: Eq a => a -> [a] -> Int`, the number of times a value appears in a list. `count 'a' "banana" == 3`

b) [3 marks] Write `interleave :: [a] -> [a] -> [a]`, which takes items from the two lists in turn; when one list runs out, the rest of the other follows. `interleave [1,3,5,7] [2,4] == [1,2,3,4,5,7]`

c) [4 marks] Write `runLengths :: Eq a => [a] -> [(a, Int)]`, which groups runs of equal neighbouring items with their length. You may use `takeWhile` and `dropWhile`. `runLengths "aaabcc" == [('a',3),('b',1),('c',2)]`

Question 4: Higher-order functions (8 marks)

a) [3 marks] Write `lengthF :: [a] -> Int` using `foldr` and no recursion.

b) [5 marks] Write `applyAll :: [a -> a] -> a -> a`, which applies every function in the list, the LAST one first. `applyAll [(+1), (*2)] 5 == 11` (5 doubled is 10, then add 1). Full marks for a one-line answer using `foldr`.

Question 5: Data types and trees (10 marks)

```
data Shape = Circle Double | Rect Double Double
data BT a  = Empty | Fork a (BT a) (BT a)
```

a) [3 marks] Write `area :: Shape -> Double`. (Use `pi`.)

b) [3 marks] Write `maxBT :: Ord a => BT a -> Maybe a`, the largest value in a tree, or `Nothing` for an empty tree. The tree is NOT a search tree. You may define a helper.

c) [4 marks] Write `paths :: BT a -> [[a]]`, every path from the root down to a fork with two empty subtrees. For `t = Fork 1 (Fork 2 Empty Empty) (Fork 3 (Fork 4 Empty Empty) Empty)`, `paths t == [[1,2],[1,3,4]]`.

Question 6: Proof by induction (6 marks)

Given

```
length []     = 0                    -- (L1)
length (x:xs) = 1 + length xs        -- (L2)

[]     ++ ys = ys                    -- (A1)
(x:xs) ++ ys = x : (xs ++ ys)        -- (A2)
```

prove by induction on `xs` that, for all lists `xs` and `ys`,

```
length (xs ++ ys) = length xs + length ys
```

State the base case, the induction hypothesis and the step case, and name the equation used at each step.